



JOMO KENYATTA UNIVERSITY OF AGRICULTURE & TECHNOLOGY

SCHOOL OF OPEN, DISTANCE AND eLEARNING

P.O. Box 62000, 00200

Nairobi, Kenya

E-mail: elearning@jkuat.ac.ke

BIT 2114 Object Oriented Analysis and Design

LAST REVISION ON April 28, 2014





This presentation is intended to be covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green **Exercise** tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elarning@jkuat.ac.ke.



LESSON 8

Interaction Diagrams

Learning outcomes

Upon completing this topic, you should be able to:

1. Describe the different interaction diagrams
2. Model the requirements from the use case diagram to all of the models examined.



8.1. Interaction Diagrams

There are two types of interaction diagrams namely:

1. Collaboration diagrams
2. Sequence diagrams

Both of which express similar information but in different ways. The sequence diagram shows the explicit sequencing of messages while collaboration diagrams shows the relationship among objects.

8.2. Sequence Diagram

It can also be defined as a dynamic model that shows the explicit sequence of messages that are passed between objects in a defined interaction because the pattern of interaction varies



BIT 2114 Object Oriented Analysis and Design

from one use case to another.

Collaboration diagrams are used to show simple interactions between objects. While it is possible to define complicated behaviour using collaborations, often the diagrams become very visually complicated and difficult to follow. Sequence diagrams are specifically suited to defining complex interactions over a period of time. It is important to remind you that collaboration and sequence diagrams can be used interchangeably.

However, each have their specific uses. Sequence diagrams show interactions between objects over time. Sequence diagrams are often preferable to collaboration diagrams when:

- There are several interactions involved in a particular behaviour. The sequence (temporal placement) of these interactions gets fairly complicated



BIT 2114 Object Oriented Analysis and Design

Consider an operation called purchase in a warehouse environment. It may involve a number of steps:

- Charge the price of the item (plus tax) to the customer's account
- Removing the product from the inventory database
- Creating a shipping order for the product when it arrives so the item will be delivered to the customer's address



Sequence Diagrams: Example

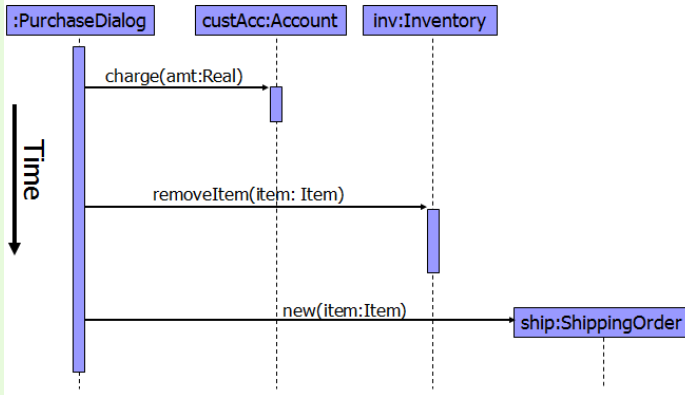


Figure 8.1: Sequence Diagram

Sequence diagrams capture the behavior of single use cases.

Hence a sequence diagram shows how objects may interact to realize a use case; each sequence diagram specifies the main in-



BIT 2114 Object Oriented Analysis and Design

teraction steps to be achieved for each use case. Use-cases are broken down into sequences of actions and events. These actions and events are mapped onto objects through object interaction diagrams.

8.2.1. Elements of a Sequence Diagram (Notations/Symbols)

1. An actor

This is a person or system that derives benefits from the system and is external to this system, who participate in a sequence by sending or receiving messages. It is placed across the top of a diagram.

1. An Object/Class:

This is an object/class or object instances that participate in sequence by sending of/or receiving messages. Actor's classes,



BIT 2114 Object Oriented Analysis and Design

objects or object instances are shown at the top of the diagram. The left most object is the starting object and maybe a person (for which a use case actor symbol is used), window or dialog box or other user interfaces. The top messages use indicators in their name to indicate who the rectangle represents. This can be shown as follows.

Object Name

: Class

Object:: Class

1. Life line

This denotes the life of an object that sequence. It presents the life for the class of an object flow when it was created through and when it is destroyed.

1. Focus of control



BIT 2114 Object Oriented Analysis and Design

This shows/represents an activation of the object of class. Activation shows the time period during which the object or class performs an operation, either directly or through a call to some subordinate operation. The bar is placed at the top of a life-line. It shows when an object is busy doing things i.e sending or receiving messages.

1. A message

This is the interaction from one object or class to another. A message conveys information from object to another one. Messages belong to the receiving class or objects. There are variations in message arrows. These are:

- Solid Arrow (Full arrow)

This represents synchronous messages. A type of message in which the caller has to wait for the receiving objects to finish



BIT 2114 Object Oriented Analysis and Design

executing the call operation before it can resume execution itself. They are used when the sending class and the control is returned to the sending class when the receiving message finishes the execution. Synchronous messages always have associated return messages. The message only may provide the caller with some return value(s) or simply back to the caller that the operation called has been successfully completed.

- Open Arrow (Half arrow)

This represents asynchronous messages. This is a message in which the sender does not have to wait for the recipient to handle the message. The sender can continue executing the message of immediately after sending the message. Asynchronous messages are common in concurrent real time system where several objects operate in parallel.



- Dashed Full Arrow

Used to show a returning message.

- Object Destruction

Used to show when an object goes out of existence. It represents when the object is destroyed. Placed at the end of an object's lifeline.

8.2.2. Message Labeling

Messages can be labeled using the following:

- The name of message followed by an empty parenthesis
e.g. message name ()
- The name of message followed by parameters in the parenthesis e.g. message name (parameter1, ———etc)



BIT 2114 Object Oriented Analysis and Design

- The name of the message followed by parameter type, parameter name and any default value for the parameter in the parenthesis e.g. message name (parameter type, parameter name, default value).
- The message may be a stereotype such as <create> indicating that a new object is created as a result of some actions.

NB:

Timing in the sequence diagram is displayed from top to bottom. The first interaction is shown at the top of the diagram and the last is showed at the bottom of a diagram. The interaction allows begins at the top of the bar of actor or object.

Sequence diagrams can be used to translate the use case scenario into a visual tool for system analysis. The initial sequence



BIT 2114 Object Oriented Analysis and Design

diagram used in system analysis shows the actors and classes in the system and interactions between them for a specific process.

Example: Consider the bank example

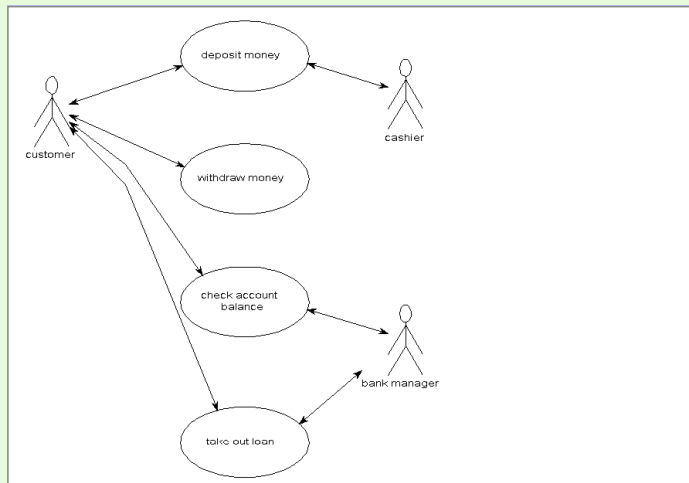




Figure 8.2: Bank Sequence Diagram

The deposit cash involves the following sequence of actions:

1. Complete pay in slip
2. Hand cash to cashier
3. Increase cash balance of account
4. Increase cashier's balance
5. Increase bank's balance
6. Issue receipt to customer

It could be more complicated if we included cheques, but you can do that as an exercise. The link between the requirements model and the object model is built through object interaction diagrams. We can now construct an object interaction diagram something like:



Complete pay-in slip

Hand pay-in slip to
cashier

Increase customer
account balance

Increase cashier balance

Increase banks overall
balance

Issue receipt to
customer



Figure 8.3: Bank Object Interaction Diagram

Now we have to consider what objects we might need for interaction. The first debate is whether we want to model the pay-in slip. In the final analysis, the decision to model something as an object depends on how important the behaviour is to the system we are developing. It might be that we only need to record the existence of the pay-in slip, and it can be referenced by an attribute. But to be safe, at first we shall include it as an object.



Complete pay-in slip
Hand pay-in slip to cashier
Increase customer account balance
Increase cashier balance
Increase banks overall balance
Issue receipt to customer

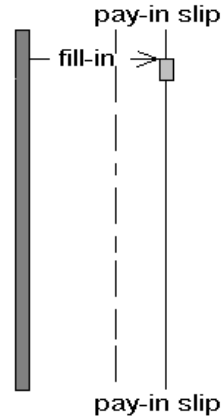


Figure 8.4: Bank Pay-in Slip

Next we have to decide if we wish to model the cashier. Again, at first it is better to model something as an object and reduce its status to an attribute (say) or remove it from the system at a later stage. So we include a second object.

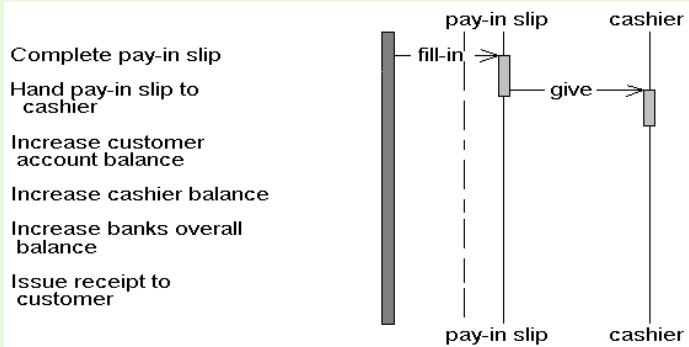


Figure 8.4: Bank Pay-in Slip Cashier Interaction

Now we have to worry about the customer and his/her account(s). Now a customer may have more than one account, normally. So it seems sensible to model both. Here, however, we merely need to know about the account. In fact the person depositing money may not be the owner of the account.

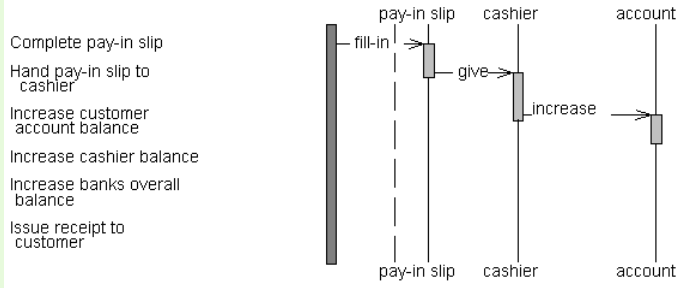


Figure 8.5: Bank Pay-in Slip Cashier Interaction

Now we are left with trying to understand where the cashier balance might be stored. We might want to create an object for this, or store this as an attribute of the cashier object. There seems no need to keep a cashier balance separate, so we shall record the cashier balance in the cashier object.

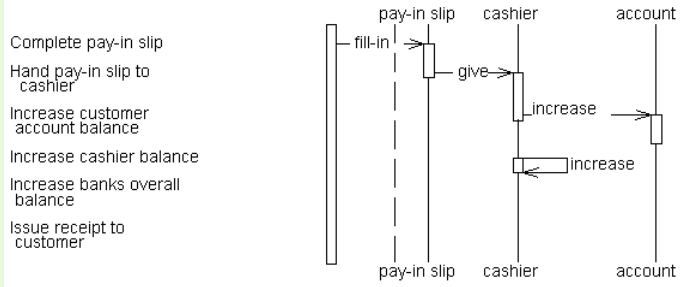


Figure 8.6: Bank Pay-in Slip Cashier Interaction

Now we are left with a decision about whether to keep a bank object. It seems reasonable, even though there may only be one. (Alternatively, we might think of the bank having many branches, and have to increment the branch balance. Let's keep it simple though.)

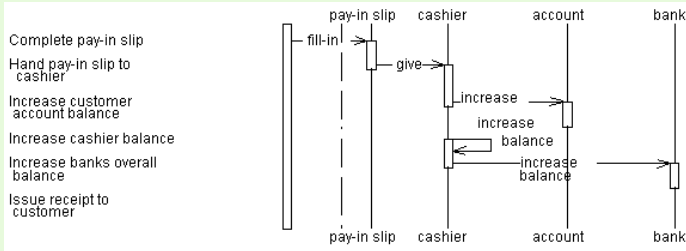


Figure 8.7: Bank Pay-in Slip Cashier Interaction

Now we have to decide whether to include the customer object. At this stage, the thing we are talking about is a real, physical customer. Hence we might leave a customer object out of the interaction diagram.

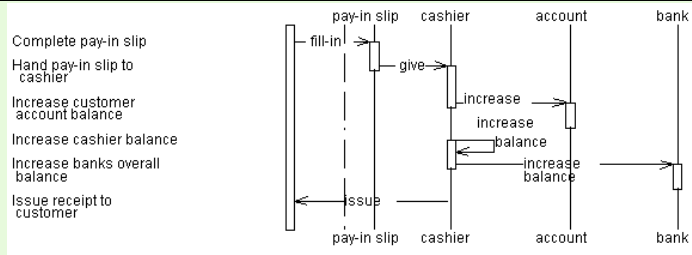


Figure 8.8: Bank Pay-in Slip Cashier Interaction

And so we progress. Now it is almost certain that two analysts working on the system would come up with different decisions at each stage. There is no single, definitive answer. Hopefully the set of objects derived by independent analysts would be similar, but any expectation that analysis is a deterministic activity with right or wrong answers is bound to be frustrated. Of course, there are good and bad analyses and designs.



BIT 2114 Object Oriented Analysis and Design

A large part of what makes for good design comes from experience. In a nutshell, the first step from requirements analysis to detailed analysis is through the use of object-interaction diagrams to define objects, their behaviour and their attributes. These objects will be found in many interaction diagrams, and their overall behaviour is a composite of the behaviours from the individual interaction diagrams. Having found some of the behaviours of objects in this way, we can then produce state-transition diagrams for the objects to more tightly define their behaviour.

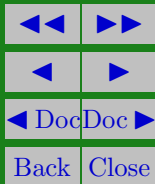
Our initial analysis can progress beyond functionality towards some architecture to support that functionality. There are many techniques for driving the analysis. An analysis should involve the appropriate parties in the development of the system.



BIT 2114 Object Oriented Analysis and Design

There is not enough scope in this course to discuss methods of interviewing. One useful technique, however is to provide some discussion around the interface of the use-cases with the actors.

This has the advantage that it begins to open up the way the system may be used, and it defines both attributes and operations the use-case needs to take into consideration. For example, suppose we wished to discuss with a bank manager what happened when he/she checked an account balance. We might draw up a possible screen and describe it to the bank manager, asking for feedback on the suitability of the screen, its contents, and what the bank manager would expect to be able to do through the screen. It can be related to the use case diagram as below.



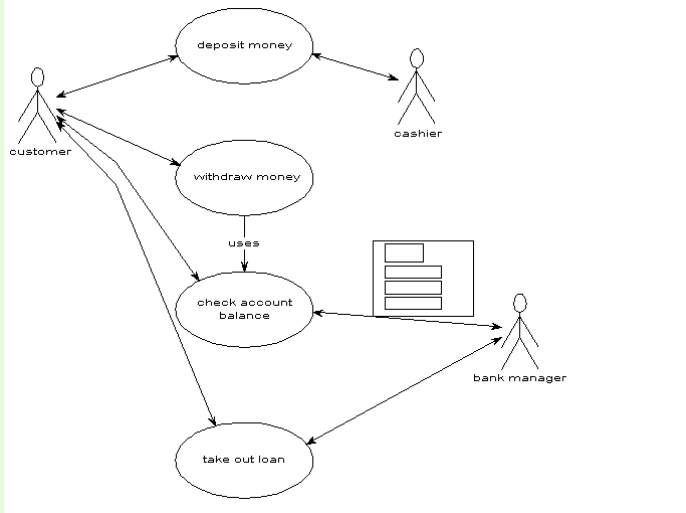


Figure 8.9: Bank Use Cases

As the analysis progresses through to design, interface objects will have to be designed to drive the interfaces with the



BIT 2114 Object Oriented Analysis and Design

various actors, and effectively implement the use-cases as objects in their own rights.

Use-cases can be thought of as objects whose behaviour is defined in terms of other objects in the system. Abstract analysis and design can progress a long way before taking into consideration the hardware and software requirements for implementation.

Arguably it is better to defer the decisions about implementation as far down the line as possible, though in practice it is often important to have some idea of the basic hardware and software configurations which will be available; in many practical projects these will be given as part of the existing infrastructure or policy of the client.

In developing sequence diagrams and consequently distribut-



BIT 2114 Object Oriented Analysis and Design

ing the responsibilities the following guidelines need to be considered:

1. Employ the use case diagram to determine the use cases. Determine the actors that interact with the use case. Don't model interactions between actors since they are outside the scope of the system
2. Consider one use case at a time. Obtain the text-based , detailed description of each use case
3. For a given use case, specify a boundary class between the use case and each actor. Thus, if there are two actors, there will be two boundary classes
4. For a given use case, specify one control class
5. Derive the entity classes from object relations for the use



case

6. Interactions in a sequence diagram are among objects of the classes, not among the classes themselves. Use the notation :Customer to indicate an object of the class Customer

8.3. Summary

Learning Activities

1. Now draw sequence diagrams for the class timetabling system. For example, the one below is the sequence diagram for LOGIN.

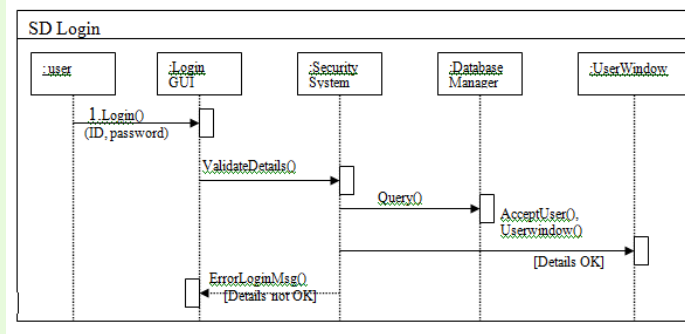


Figure 8.10: Login Sequence Diagram

8.4. Collaboration Diagram

Structured from the perspective of interactions among objects. It captures dynamic behavior (Message-oriented).

Its purposes is to:



BIT 2114 Object Oriented Analysis and Design

- Model flow of control
- Illustrate coordination of object structure and control
- Objects that interact with other objects
- Sequence::Time vs. Collaboration::Message

Main Concepts: Collaboration, Interaction, Collaboration Role, Message.

Collaboration diagram captures dynamic behavior (Message-Oriented)

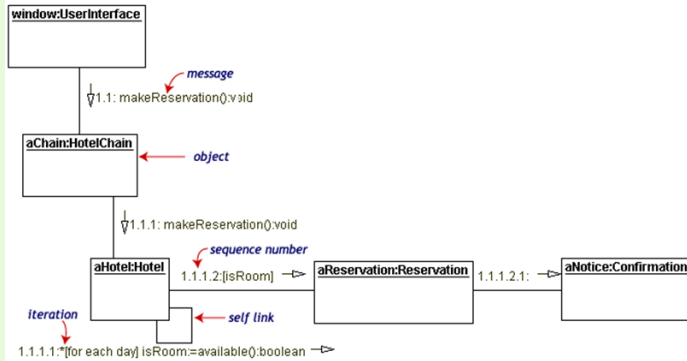


Figure 8.11: Collaboration Diagram

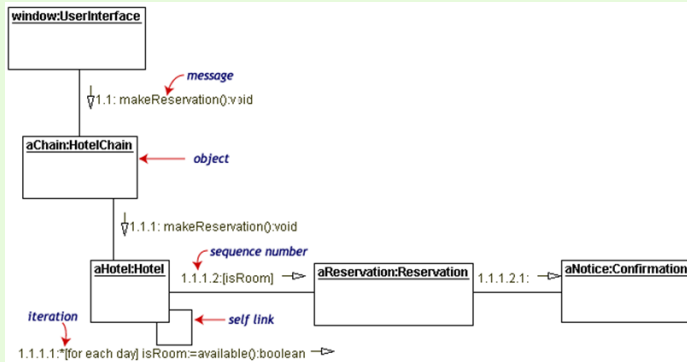


Figure 8.12: Collaboration Diagram

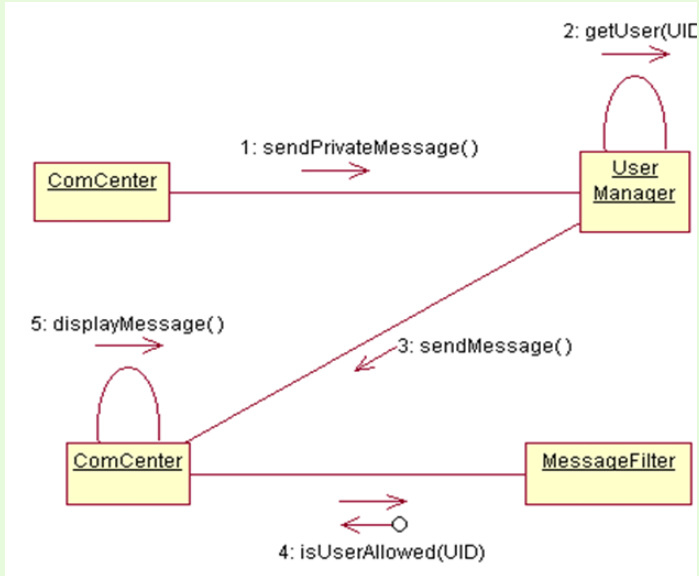


Figure 8.13: Collaboration Diagram



Learning Activities

Explain how use cases and class diagrams are related to interaction diagrams.

Revision Questions

Example . Explain the difference between a state diagram and an activity diagram

Solution: for revision



EXERCISE 1.  Explain the elements that make up an activity diagram

EXERCISE 2.  Create a state diagram for the following scenario concerning a Car class A car can be booked via an order,



JKUAT SODEL
©2014

BIT 2114 Object Oriented Analysis and Design

if the date is free. A Car can be either in Store or on Loan depending on the bookings. A car can be booked for a future date whether it is on loan or not.

◀◀	▶▶
◀	▶
◀ Doc	Doc ▶
Back	Close



References and Additional Reading Materials

1. Richard Lee , William M.Tepfenhart (2000). UML and C++ - A practical guide to object oriented development” , ISBN-13: 978-0130290403
2. John. W Satzinger (2004). Object Oriented Analysis and Design with the Unified Process. Cengage Learning, ISBN-13: 978-0619216436
3. Annals Of software Engineering ISSN 1022-7091